Vercel AI SDK abstraction layer simplifies interactions with AI models but can introduce risks if developers are not fully aware of its underlying mechanics. Misconfiguration or incorrect assumptions about SDK behavior can lead to errors. The LangChain.js framework comes with its own set of challenges. The fast code iterations and changes of this library mean that best practices are continually changing, so developers must keep up with the new norms.

As always, we can improve the stability and security of our software by adhering to good engineering practices and implementing thorough testing and debugging strategies. Here we will explore essential testing and debugging techniques specifically tailored for the technologies we use in this book. You can, of course, apply the key concepts to any framework or software.

This chapter assumes familiarity with the technologies introduced in previous chapters (Next.js, Vercel AI SDK, LangChain.js) and experience implementing AI-driven features. You should be comfortable navigating React and server-side code, understanding SDK interactions, and integrating AI workflows.

8.1 Debugging Next.js AI applications

Next.js is a powerful full-stack framework for building React applications. When integrating AI—whether through direct large language model (LLM) APIs or the Vercel AI SDK—we introduce unique debugging challenges due to the complex architecture that involves server-side AI processing, streaming responses, and nontrivial state management.

While no universal debugging strategy applies to all scenarios, we can develop a systematic approach based on identifying and analyzing symptoms. For instance, when a user submits a prompt and encounters an error, we can follow a structured debugging process to pinpoint the root cause and implement an appropriate solution.

The three main problem areas for Next.js are rendering, state management, and performance. Figure 8.1 outlines key steps to debugging these kinds of problems. Let's go through the problems and solutions represented in the figure.

8.1.1 Debugging common Next.js rendering Issues

By default, Next.js provides some excellent debugging features out of the box. If the React component code logic is wrong or throws an error, it will produce an overlay that covers the page and highlights the error. Figure 8.2 shows an example of this overlay.

> **NOTE** The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch08/chat-rendering-troubleshooting.

When an error is thrown during the rendering cycle, Next.js presents an overlay containing the call stack as soon as the page is loaded.